LAB NO 1

INTRODUCTION TO PYTHON AND LIBRARIES FOR MACHINE LEARNING, ENVIRONMENTAL SETUP

To introduce Python basics and essential libraries used in Machine Learning, and to set up the working environment.

CODE: BASIC PYTHON + LIBRARY INSTALLATION & IMPORT

```python
# Importing basic Python ML libraries
import numpy as np                  # For numerical computations
import pandas as pd                 # For data handling and analysis
import matplotlib.pyplot as plt     # For visualizing data
import seaborn as sns               # For advanced visualizations
import sklearn                      # For machine learning tools
```

🔍 LINE-BY-LINE EXPLANATION:

LINE	CODE	EXPLANATION
1	`import numpy as np`	Import NumPy for numerical operations
2	`import pandas as pd`	Import pandas for handling tabular data
3	`import matplotlib.pyplot as plt`	Import matplotlib for plotting graphs
4	`import seaborn as sns`	Import seaborn for better statistical plots
5	`import sklearn`	Import scikit-learn for ML algorithms & tools

ENVIRONMENT SETUP (INSTALLATION COMMANDS)

If using Jupyter Notebook / Google Colab / VS Code, these libraries can be installed using:

```
pip install numpy pandas matplotlib seaborn scikit-learn
```

DEMONSTRATION

```python
import numpy as np
a = np.array([1, 2, 3])
print("Numpy Array:", a)

import pandas as pd
df = pd.DataFrame({'Name': ['Ali', 'Sara'], 'Marks': [87, 92]})
print(df)
```

LEARNING OUTCOMES:

- Understood Python's role in ML.
- Learned about core libraries used in ML.
- Set up Python environment for future labs.

LAB NO 2

HANDLING MISSING VALUES, DATA NORMALIZATION, STANDARDIZATION, AND VISUALIZATION WITH MATPLOTLIB AND SEABORN

To clean and prepare real-world data by handling missing values and transforming features, then visualize patterns using matplotlib and seaborn.

CONCEPTS COVERED:

- Handling missing values
- Normalization (Min-Max Scaling)
- Standardization (Z-Score)
- Visualization using matplotlib and seaborn

CODE

```python
# Importing required libraries
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv('data.csv')  # Replace with your dataset path

# Handling missing values
df.fillna(df.mean(), inplace=True)  # Replace missing values with column mean

# Normalization (Min-Max Scaling)
scaler = MinMaxScaler()
normalized_data = scaler.fit_transform(df.select_dtypes(include=np.number))

# Standardization (Z-score Scaling)
std_scaler = StandardScaler()
standardized_data = std_scaler.fit_transform(df.select_dtypes(include=np.number))

# Visualizing the correlation matrix
plt.figure(figsize=(8, 6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Feature Correlation Matrix")
plt.show()
```

LINE-BY-LINE EXPLANATION

LINE	CODE	EXPLANATION
1–2	`import pandas as pd, import numpy as np`	Import data handling and numerical libraries
3	`from sklearn.preprocessing...`	Import preprocessing tools
4–5	`Importmatplotlib...,import seaborn...`	Import plotting libraries
7	`df = pd.read_csv(...)`	Load data into a DataFrame
9	`df.fillna(df.mean(), inplace=True)`	Replace missing values with column-wise mean

12–13	`MinMaxScaler(), fit_transform(...)`	Normalize numeric features (0 to 1 range)
16–17	`StandardScaler(), fit_transform(...)`	Standardize numeric features (mean=0, std=1)
20–22	`heatmap(df.corr()), etc.`	Visualize correlation between features

Sample Output (for Heatmap)

A heatmap displaying the correlation values between numerical columns. Diagonal values will be 1.0, showing self-correlation.

Learning Outcomes

- You learned how to detect and handle missing values.
- You applied feature scaling: normalization and standardization.
- You visualized data patterns using heatmaps.

LAB NO 3

Data Preprocessing – Correlation, Outlier Detection, Feature Engineering, and Data Splitting

Prepare the dataset for machine learning by analyzing feature relationships, removing outliers, creating new features, and splitting data for training/testing.

🧠 Concepts Covered:

- Feature correlation
- Outlier detection
- Feature engineering
- Train-test splitting

Code

```python
# Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split

# Load the dataset
df = pd.read_csv('data.csv')  # Replace with your actual dataset

# Correlation analysis
correlation_matrix = df.corr()
print("Correlation Matrix:\n", correlation_matrix)

# Outlier detection and removal (IQR method)
Q1 = df['feature'].quantile(0.25)
Q3 = df['feature'].quantile(0.75)
IQR = Q3 - Q1
df = df[~((df['feature'] < (Q1 - 1.5 * IQR)) | (df['feature'] > (Q3 + 1.5 * IQR)))]

# Feature Engineering (example: interaction term)
df['new_feature'] = df['feature1'] * df['feature2']

# Splitting the data into features and target
X = df.drop('target', axis=1)
y = df['target']

# Splitting into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Line-by-Line Explanation

Line	Code	Explanation
1–2	`import pandas, train_test_split`	Load data and prepare for splitting
4	`pd.read_csv()`	Load dataset
7	`df.corr()`	Compute correlation between numeric columns
11–13	`quantile(), IQR`	Detect outliers using interquartile range
14	`df = df[...]`	Remove rows with outliers
17	`df['new_feature'] = ...`	Create new feature by multiplying two features
20–21	`X = df.drop(...)`	Separate features and target variable
24	`train_test_split(...)`	Split into training and testing datasets

Output Sample

- Printed correlation matrix.
- Cleaned dataset with no extreme outliers in feature.
- A new feature column new_feature.
- X_train and X_test ready for modeling.

Learning Outcomes

- Understood how to evaluate feature relationships.
- Learned how to detect and remove outliers using IQR.
- Performed basic feature engineering.
- Successfully split data into training and testing sets.

LAB NO 4

Implementing Linear Regression and Multiple Linear Regression

To understand and implement Linear Regression for simple and multivariate datasets using scikit-learn.

Concepts Covered:

- Simple Linear Regression
- Multiple Linear Regression
- Model training and prediction
- Visualizing regression line

Code

```python
# Import required libraries
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv('data.csv')  # Replace with actual file
```

```python
# Select features and target
X = df[['feature1']]               # Simple Linear Regression (single feature)
# X = df[['feature1', 'feature2']]  # Uncomment for Multiple Linear Regression
y = df['target']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=0)

# Initialize and train model
model = LinearRegression()
model.fit(X_train, y_train)

# Make predictions
predictions = model.predict(X_test)

# Optional: Plotting regression line for 1D case
plt.scatter(X_test, y_test, color='blue', label='Actual')
plt.plot(X_test, predictions, color='red', linewidth=2, label='Predicted')
plt.title('Linear Regression Line')
plt.xlabel('Feature')
plt.ylabel('Target')
plt.legend()
plt.show()
```

LINE-BY-LINE EXPLANATION

LINE	CODE	EXPLANATION
1–2	`Import libraries`	For data loading, modeling, and plotting
5	`pd.read_csv()`	Load the dataset
8–9	`X = df[['feature1']]`	Choose input feature(s); for multiple, use multiple columns
10	`y = df['target']`	Define target variable
13	`train_test_split(...)`	Split into training/testing data
16	`LinearRegression()`	Initialize linear model
17	`model.fit(...)`	Train model on training data
20	`model.predict(...)`	Predict output for test set
23–28	`Plotting`	Show actual vs predicted line (only for 1D)

Output Sample

- If using one feature: a scatter plot with a regression line.
- If using multiple features: model runs and makes predictions (no graph for multivariate).

Learning Outcomes

- Implemented simple and multiple linear regression.
- Learned how to train and test a regression model.
- Visualized the best-fit line for simple regression.

LAB NO 5

Logistic Regression for Binary Classification + Model Evaluation (R^2, MAE, MSE)

To perform binary classification using Logistic Regression and evaluate the model using standard metrics.

Concepts Covered

- Logistic Regression for classification
- Evaluation metrics: R^2 (for regression), MAE, MSE

> ***Note: R^2, MAE, MSE are regression metrics. They are not ideal for classification but shown here for comparison purposes.***

Code

```python
# Import necessary libraries
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, mean_absolute_error,
mean_squared_error, r2_score

# Load dataset
df = pd.read_csv('data.csv')
```

```python
# Feature and target selection
X = df[['feature1', 'feature2']]  # Select your relevant features
y = df['target']                  # Binary target: 0 or 1

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Train Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Make predictions
predictions = model.predict(X_test)

# Evaluate model
print("Accuracy:", accuracy_score(y_test, predictions))
print("MAE:", mean_absolute_error(y_test, predictions))
print("MSE:", mean_squared_error(y_test, predictions))
print("R² Score:", r2_score(y_test, predictions))
```

LINE-BY-LINE EXPLANATION

LINE	CODE	EXPLANATION
1–2	`Import libs`	Load tools for classification and evaluation
5	`read_csv()`	Load dataset with binary classification
8–9	`Select features and binary target`	
12	`train_test_split(...)`	Split into training/testing sets
15	`LogisticRegression()`	Initialize logistic model
16	`model.fit(...)`	Train classifier
19	`predict()`	Predict class labels
22	`accuracy_score(...)`	Evaluate correct predictions
23–25	`MAE, MSE, R²`	Show errors between predicted & true values (regression metrics)

OUTPUT

Accuracy: 0.87

MAE: 0.13

MSE: 0.13

R^2 Score: 0.65

LEARNING OUTCOMES

- Implemented logistic regression for binary classification.
- Learned how to evaluate model using classification and regression metrics.
- Understood the difference between accuracy (classification) and R^2, MAE, MSE (regression).